

Multi-node JAX on MIT Supercloud

A Practical Guide for PeriodicWave Users

PeriodicWave

2026-06-22

Contents

1	Overview	2
2	Required code changes	2
2.1	SLURM submission script	2
2.2	Runner script: three additions	3
2.3	Checkpoint restore	5
3	The five critical bugs	5
4	Files that do not change	7
5	Batch size semantics	8
6	Verification checklist	8
7	Open issues	8

1 Overview

This document is a complete implementation recipe for running PeriodicWave VMC across multiple nodes on MIT Supercloud. **It records every bug encountered, in order, with exact symptoms, root causes, and fixes** — so a future implementer can get to a working multinode run without hitting any of them.

Why the training code needs no changes

PeriodicWave uses `jax.pmap` for all parallelism. Once `jax.distributed.initialize()` is called, `pmap` automatically spans all GPUs across all nodes. Every collective in `periodicwave/constants.py` uses `lax.pmean` / `lax.psum` with `axis_name='qmc_pmap_axis'`, which already performs cross-host NCCL communication when JAX is initialised with distributed support.

The training loop, loss, MCMC, KFAC, and network are unchanged. Only the runner script, the SLURM script, and the checkpoint module need modification.

Gradient computation is fully global

The mean local energy `loss = pmean(mean(e_l))` averages over *all* walkers across all nodes before forming `diff = e_l - loss` and computing the gradient. The gradient is then reduced again via `grad = pmean(grad)`. The result is mathematically identical to a single-machine run with N_{GPU} times more walkers.

Supercloud GPU topology

Each Supercloud node in the `volta` partition has **2 Tesla V100-PCIE-32GB** GPUs. A typical multinode job uses $N_{nodes} \times 2$ total processes, one per GPU.

2 Required code changes

2.1 SLURM submission script

Create a dedicated script (e.g. `run_multinode.sh`).

```
#!/bin/bash
#SBATCH --nodes=2                # default; overridden by sbatch --nodes=N
#SBATCH --ntasks-per-node=2     # GPUs per node (volta: 2)
#SBATCH --gres=gpu:volta:2
#SBATCH --cpus-per-task=4
#SBATCH --mem=48G
#SBATCH --time=24:00:00
#SBATCH --exclusive
#SBATCH --output=/home/gridsan/%u/.../jobs/jell-%j.out
#SBATCH --error=/home/gridsan/%u/.../jobs/jell-%j.out

set -euo pipefail
set +u; source /etc/profile; set -u

module load anaconda/Python-ML-2025a
source "$HOME/venv/periodicwave/bin/activate"
```

```

export JAX_ENABLE_X64=1
export JAX_PLATFORM_NAME=gpu
export XLA_PYTHON_CLIENT_PREALLOCATE=false
export XLA_PYTHON_CLIENT_MEM_FRACTION=0.90

# Bug 4: job-specific port avoids stale coordinator conflicts
PORT=$((29500 + SLURM_JOB_ID % 10000))
COORDINATOR="$(scontrol show hostname "$SLURM_NODELIST" | head -1):${PORT}"
echo "COORDINATOR=${COORDINATOR} NTASKS=${SLURM_NTASKS}"

# Bugs 2 & 3: use SLURM env vars; --gpus-per-task=1 for correct GPU binding
srun \
  --nodes="$SLURM_NNODES" \
  --ntasks="$SLURM_NTASKS" \
  --ntasks-per-node="$SLURM_NTASKS_PER_NODE" \
  --gpus-per-task=1 \
  --distribution=block:block \
  python -u your_runner.py \
    --coordinator_address "$COORDINATOR" \
    --num_processes      "$SLURM_NTASKS" \
    "$@"
# NOTE: do NOT pass --process_index here (Bug 2)

```

Using SLURM env vars in `srun` means the script scales automatically when you override the SBATCH header at submission:

```

sbatch --nodes=6 --ntasks=12 --ntasks-per-node=2 run_multinode.sh [args...]

```

2.2 Python runner: additions required

A — Module-level distributed pre-init (Bug 1)

Place this at the **very top of the file**, before any `import jax` statement.

```

import argparse, os

def _distributed_preinit():
    _p = argparse.ArgumentParser(add_help=False)
    _p.add_argument("--num_processes", type=int, default=1)
    _p.add_argument("--coordinator_address", default=None)
    _p.add_argument("--process_index", type=int, default=None)
    _a, _ = _p.parse_known_args()
    if _a.num_processes > 1:
        import jax as _jax
        process_id = _a.process_index
        if process_id is None:
            process_id = int(os.environ.get('SLURM_PROCID', 0)) # Bug 2
        _jax.distributed.initialize(
            coordinator_address = _a.coordinator_address,
            num_processes       = _a.num_processes,
            process_id          = process_id,
            local_device_ids    = [0], # Bug 3
        )

```

```

_distributed_preinit()    # BEFORE any other jax import

import jax
import jax.numpy as jnp
# ... rest of imports

```

Add the three CLI arguments to your parser:

```

mn = parser.add_argument_group("distributed")
mn.add_argument("--coordinator_address", default=None)
mn.add_argument("--process_index",      type=int, default=None)
mn.add_argument("--num_processes",      type=int, default=1)

```

B — Rank-0 guard on all file I/O (Bugs 5, 6, 10, 11)

Set `is_rank0` immediately after JAX is available, then guard *every* file write, hook installation, and subprocess launch:

```

is_rank0 = (jax.process_index() == 0)

if is_rank0:
    # config.json, run_args.json, device_info.log, lattice files,
    # extension_info.json (warm-start), post-process hook, ...
    _install_post_process_hook(folder, jobid)

# Suppress train_stats.csv writes on non-rank-0 (Bug 6)
class _NullWriter:
    class _NullFile:
        def flush(self): pass
    _file = _NullFile()    # satisfies writer._file.flush()
    def __enter__(self): return self
    def __exit__(self, *a): pass
    def write(self, t, **kwargs): pass

writer_manager = None if is_rank0 else _NullWriter()
train.train(cfg, writer_manager=writer_manager)

```

`train.py` creates the real writers. `Writer` only when `writer_manager` is `None`, so `_NullWriter` bypasses CSV creation entirely.

C — Deterministic output folder name (Bug 8)

Each process generates its own timestamp; processes starting in different seconds produce different folder names and create separate directories.

```

if args.jobid and args.num_processes > 1:
    tag = f"run_jobid{args.jobid}"          # identical across all ranks
else:
    tag = f"run_{timestamp}_{args.jobid}"  # single-node: fine

```

D — Patch `periodicwave.checkpoint` at runtime (Bugs 9, 12)

`train.py` calls both `checkpoint.save` (periodic checkpoints) and `checkpoint.restore` (pre-emption recovery) directly on `periodicwave.checkpoint`. Both must be replaced with multiprocess-aware versions before calling `train.train()`:

```
if args.num_processes > 1:
    import periodicwave.checkpoint as _pc
    _pc.save      = checkpoint.save      # your aggregating version
    _pc.restore   = checkpoint.restore   # your splitting version
```

This must come *after* any post-process hook installation, so rank 0 picks up the hooked version of `checkpoint.save`.

2.3 Checkpoint module: aggregating save and splitting restore

Shadow `periodicwave/checkpoint.py` with a local module rather than modifying the shared file.

Save — gather all walkers to rank 0

`process_allgather` is a Python-level collective from `jax.experimental.multihost_utils` that works *outside* of `pmap`. All processes call it simultaneously; every process receives the full stacked result.

```
from periodicwave.checkpoint import save as _core_save

def save(filename, t, data, params, opt_state, mcmc_width):
    if jax.process_count() > 1:
        from jax.experimental.multihost_utils import process_allgather
        local_pos = np.array(data.positions) # (local_devs, batch, Ne, ndim)
        all_pos   = process_allgather(local_pos)
        # shape: (num_procs, local_devs, batch, Ne, ndim)
        s = all_pos.shape
        total_walkers = s[0] * s[1] * s[2]
        global_pos = all_pos.reshape(1, total_walkers, *s[3:])
        data = data.replace(positions=global_pos) # NOTE: .replace not ._replace (Bug
12)
    if jax.process_index() == 0:
        _core_save(filename, t, data, params, opt_state, mcmc_width)
```

The saved checkpoint always contains all walkers. `params`, `opt_state`, and `mcmc_width` are identical across processes (kept in sync by `pmean`) so rank 0's copies are authoritative.

Restore — split walkers per rank

```
def restore(restore_filename, batch_size=None):
    # ... load data with np.load ...
    stored      = data.positions
    total_w     = stored.shape[0] * stored.shape[1]
    local_n     = jax.local_device_count()
    n_proc      = jax.process_count()

    if n_proc > 1:
```

```

per_proc = total_w // n_proc
rank      = jax.process_index()
flat      = stored.reshape(total_w, *stored.shape[2:])
local     = flat[rank * per_proc : (rank + 1) * per_proc]
data      = data.replace(
    positions=local.reshape(local_n, per_proc // local_n, *stored.shape[2:]))
else:
    # single-process: validate local_device_count (not global)
    if stored.shape[0] != local_n:          # Bug 9 fix
        raise ValueError(...)

if batch_size is not None:
    actual = data.positions.shape[0] * data.positions.shape[1]
    if actual != batch_size:
        raise ValueError(...)
return t, data, params, opt_state, mcmc_width

```

A checkpoint from an N -GPU run can be restored on any M GPUs as long as `total_walkers` is divisible by M .

2.4 Batch size constraint (Bug 7)

`train.py` treats `cfg.batch_size` as the *total* walker count across all GPUs and divides by the total process count:

```

host_batch_size = cfg.batch_size // num_hosts    # per process
device_batch_size = host_batch_size // num_devices # per GPU

```

`batch_size` must be divisible by the *total* GPU count ($nodes \times GPUs/node$). For 12 GPUs, 1024 fails ($1024 \bmod 12 \neq 0$); use 1200, 2400, 3600, ...

The `ke_batch_size` parameter chunks the kinetic energy evaluation. Setting it equal to `device_batch_size` eliminates chunking overhead (one forward pass per device):

$$ke_batch_size = \frac{batch_size}{totalGPUs}$$

3 All twelve bugs, in order of discovery

Bug 1 — JAX distributed init must precede every JAX import

Symptom: “*JAX distributed must be initialized before the JAX backend is initialized*”, or processes silently use only local GPUs.

Cause: `import jax.numpy` triggers XLA backend init. Module-level JAX calls (e.g. dataclass field defaults like `jnp.array([])`) do the same. `main()` is called after the module is fully imported, so placing the init there is always too late.

Fix: module-level pre-parser using only `stdlib argparse`, followed by `jax.distributed.initialize()`, before any real JAX import.

Bug 2 — \$SLURM_PROCID expands to 0 in the outer bash script

Symptom: *ALREADY_EXISTS: newer incarnation* — every task tries to be the coordinator.

Cause: `SLURM_PROCID` is set by `srun` inside each child process, not in the outer script. Expanding it in bash gives 0 for all tasks.

Fix: omit `--process_index` from the `srun` command. Read `os.environ.get('SLURM_PROCID', 0)` in Python.

Related: explicit `--ntasks` in the SBATCH header alongside `--nodes` and `--ntasks-per-node` triggers the `-mpack` heuristic, packing all tasks on one node. Fix: specify the full topology only on the `srun` line using SLURM env vars.

Bug 3 — Manual CUDA_VISIBLE_DEVICES fights SLURM GPU binding

Symptom 1: *CUDA_ERROR_INVALID_DEVICE on CUDA:2* — JAX tries to open a nonexistent device ordinal.

Symptom 2 (attempted fix): *No visible CUDA-capable device detected* for local rank 1.

Cause: without `--gpus-per-task=1`, all tasks on a node share `CUDA_VISIBLE_DEVICES=0,1`. Manual override with `CUDA_VISIBLE_DEVICES=$SLURM_LOCALID` fails because Supercloud assigns GPUs by UUID, not integer index. Setting an integer over a UUID assignment makes CUDA find no devices.

Fix: `--gpus-per-task=1` in `srun`. SLURM assigns exactly 1 GPU per task and remaps it to ordinal 0. Use `local_device_ids=[0]` in Python.

Bug 4 — Port conflict with stale coordinator from a killed job

Symptom: init failure on re-submission.

Cause: zombie coordinator from a killed prior job holds the fixed port open.

Fix: `PORT=$((29500 + SLURM_JOB_ID % 10000))`.

Bug 5 — All processes race on setup file writes

Symptom: *FileNotFoundError* on `os.rename()`; coordination barrier timeout 5 minutes later (*DEADLINE_EXCEEDED, 1/4 tasks reached barrier*).

Cause: all N processes try to rename `device_info.log`, `config.json`, etc. to versioned names. The first succeeds; the rest fail because the original is already gone. One process dies silently; the JAX coordination service eventually times out.

Fix: guard all setup file I/O with `if jax.process_index() == 0`.

Bug 6 — train_stats.csv written by all processes simultaneously

Symptom: garbled CSV — interleaved rows, step numbers repeated, numbers split mid-digit (e.g. 0002 on its own line).

Cause: all N processes open the same `train_stats.csv` and call `write()` and `flush()` concurrently. OS-level interleaving corrupts the file.

Fix: pass `_NullWriter` to non-rank-0 processes (see Section 2.2B). `_NullWriter` must expose a `_file.flush()` no-op because `train.py` calls `writer._file.flush()` unconditionally.

Bug 7 — Batch size not divisible by total GPU count

Symptom: *ValueError: Batch size must be divisible by number of devices, got batch size 1024 for 12 devices.*

Cause: `train.py` divides `cfg.batch_size` by the total process count. 1024 is divisible by 4 but not by 12.

Fix: choose a batch size divisible by the total GPU count.

Bug 8 — Per-process timestamp produces divergent folder names

Symptom: multiple output directories per job (one per process that started in a different wall-clock second).

Cause: each process calls `datetime.now().strftime(...)` independently. Processes starting in different seconds get different names.

Fix: use the SLURM job ID as the folder identifier and omit the timestamp for multinode runs.

Bug 9 — `train.py` also calls `checkpoint.restore`

Symptom: crash on pre-emption recovery (existing checkpoint in save path) even though fresh-start path worked.

Cause: `train.py` calls `checkpoint.restore` directly to resume from an existing checkpoint. We patched `_pc.save` but forgot `_pc.restore`, so the original `periodicwave.checkpoint.restore` (which checks `jax.device_count()` and does not split aggregated walkers) remained in effect.

Fix: patch both: `_pc.save` and `_pc.restore`.

Bug 10 — `extension_info.json` written by all processes on warm-start

Symptom: file race on `extension_info.json` when `--restore_path` is set.

Cause: the warm-start lineage record lacked an `is_rank0` guard.

Fix: if `is_rank0`: `json.dump(..., open("extension_info.json"))`.

Bug 11 — Post-process hook launches on all processes

Symptom: N copies of `post_process.py` launch after each checkpoint, racing on the same output directory.

Cause: `_install_post_process_hook` patches `jell_slab.checkpoint.save`. We then copy that patch into `periodicwave.checkpoint.save`. All processes receive the subprocess-launching version.

Fix: guard hook installation with `if is_rank0`. Ordering matters: install hook on rank 0 first, *then* do `_pc.save = checkpoint.save`. Rank 0 picks up the hooked version; other ranks pick up the plain aggregating version. All ranks still call `process_allgather` inside `save()` for the collective.

Bug 12 — FermiNetData uses .replace(), not ._replace()

Symptom: *AttributeError: 'FermiNetData' object has no attribute '_replace'. Did you mean: 'replace'?*

Cause: FermiNetData is a Flax struct (`flax.struct.dataclass`), not a Python `namedtuple`. Flax structs use `.replace()` without a leading underscore.

Fix: replace every `data._replace(...)` with `data.replace(...)`.

4 Files that do not change

File	Reason
<code>periodicwave/constants.py</code>	<code>lax.pmean/psum</code> already cross-host after init
<code>periodicwave/train.py</code>	Sharding via <code>local_device_count</code> ; reductions correct
<code>periodicwave/loss.py</code>	All reductions through <code>constants.pmean / all_gather</code>
<code>periodicwave/mcmc.py</code>	<code>constants.pmean</code> on acceptance rate correct cross-host
<code>periodicwave/networks.py</code>	Pure forward pass, no collectives
KFAC	Initialised with <code>pmap_axis_name</code> ; multihost-aware
<code>periodicwave/checkpoint.py</code>	Shadowed by local module; never modified directly

5 Verification checklist

1. Single-process smoke test (`--num_processes 1`): run behaves identically to before changes.
2. All processes print the same global device list `JAX devices: [CudaDevice(id=0), ..., id=N-1]`.
3. SLURM log shows tasks split across distinct nodes.
4. Exactly one output folder created (job-ID name, no timestamp).
5. `train_stats.csv`: monotonically increasing step numbers, no interleaved rows.
6. Post-process hook launches exactly once per checkpoint.
7. Checkpoint file contains all walkers (`positions.shape = (1, total_walkers, Ne, ndim)`).
8. Checkpoint restore on a different GPU count succeeds.
9. Energy converges to expected value vs. single-node at same total batch.

6 Remaining open questions

- **Inter-node bandwidth:** NCCL collectives confirmed working; bandwidth not measured. Email supercloud@mit.edu to confirm InfiniBand is in use between volta nodes.
- **Cross-count checkpoint restore:** the design supports it (split by `jax.process_count()`), but end-to-end test pending.
- **Scaling beyond 6 nodes:** implementation is node-count-agnostic (SLURM env vars throughout), but untested at higher counts.

- **MCMC width synchronisation:** each process adapts `mcmc_width` via `pmean(pmove)`. Widths should stay in sync since `pmove` is a global average, but this has not been explicitly verified on the multinode path.